Program - 1

Aim:

To write a C++ program to accept and display the basic details of a user (Name, Roll No, Division and Address).

Theory:

C++ uses input and output streams to communicate with the user. The header `<iostream>` provides `cin` for input and `cout` for output. The `string` class, available through `<string>`, stores text such as names and addresses. The extraction operator `»` reads a value up to whitespace, whereas `getline()` reads an entire line, including spaces. When formatted input is followed by `getline()`, the newline left in the input buffer should be removed using `cin.ignore()` so that the next line is read correctly.

The program also demonstrates why the input method must match the type of data being entered: formatted extraction is suitable for a numeric roll number, while line-based input preserves spaces in names or addresses. Clear prompts and labelled output make the program easier to use and verify.

Algorithm:

Step - 1 Start the program and enter the main function.
Step - 2 Declare string variables for the name, division and address, and an integer variable for the roll number.
Step - 3 Prompt the user to enter the name and read the complete line using `getline()`.
Step - 4 Ask for the roll number and read it using `cin`; clear the pending newline with `cin.ignore()` before reading further lines.
Step - 5 Read the division and full address with `getline()` so that spaces in the address are preserved.
Step - 6 Print each stored value with a clear label, checking that the details appear in the intended order.
Step - 7 End the program.

Code:

```cpp
#include <iostream>
#include <string>
using namespace std;
int main()
{
    string name, division, address;
    int roll;
    cout << "Enter Name: ";
    getline(cin, name);
    cout << "Enter Roll No: ";
    cin >> roll;
    cin.ignore();
    cout << "Enter Division: ";
    getline(cin, division);
    cout << "Enter Address: ";
    getline(cin, address);
    cout << "\n------ Student Details ------";
    cout << "\nName     : " << name;
    cout << "\nRoll No  : " << roll;
    cout << "\nDivision : " << division;
    cout << "\nAddress  : " << address;
    return 0;
}
```

Output:

```
Enter Name: Dhruvansh Panchal
Enter Roll No: 25EC417
Enter Division: D-09
Enter Address: Karamsad

------ Student Details ------
Name     : Dhruvansh Panchal
Roll No  : 25EC417
Division : D-09
Address  : Karamsad
```

Code Explanation:

The program includes the input/output and string headers, then declares variables for the name, division, address and roll number. It uses ‘getline()‘ for text fields so spaces are retained, and ‘cin‘ for the numeric roll number. ‘cin.ignore()‘ removes the leftover newline before the next line-based input. Finally, labelled ‘cout‘ statements display the details entered by the user.

Conclusion:

The program accepted and displayed the user’s name, roll number, division and address. Using `getline()` allowed text containing spaces to be read correctly, while `cin.ignore()` handled the newline left after numeric input. This practical demonstrated basic console input and output in C++.

Program - 2

Aim:

To write a C++ program to perform simple arithmetic operations (addition, subtraction, multiplication, division and modulus) using the switch-case statement.

Theory:

The `switch` statement selects one block from several alternatives by comparing an expression with constant `case` labels. It is useful when a single variable, such as an arithmetic operator, determines the operation to perform. The `break` statement prevents execution from continuing into the next case, while `default` handles an unrecognised operator. The arithmetic operators `+`, `-`, `*`, `/` and `%` perform addition, subtraction, multiplication, division and remainder respectively. Division and remainder operations must not use zero as the divisor.

The selection is controlled by the operator entered by the user, so only the matching case should execute. Integer operands are used in this example; converting the numerator to a floating-point type for division allows a fractional quotient, while modulus is defined for integer operands.

Algorithm:

Step - 1 Start the program and enter the main function.
Step - 2 Declare two integer variables, `num1` and `num2`, and a character variable `op` for the selected operator.
Step - 3 Prompt for and read the first number, the second number, and one operator from `+`, `-`, `*`, `/` or `%`.
Step - 4 Evaluate `op` using `switch`; in each case calculate the corresponding arithmetic result.
Step - 5 Before division or modulus, test whether `num2` is zero; if it is, print an explanatory error instead of performing the operation.
Step - 6 Use `break` after each valid case and use `default` to report an unsupported operator.
Step - 7 Display the selected operation's result or its error message, then terminate.

Code:

```cpp
#include <iostream>
using namespace std;
int main()
{
    int num1, num2;
    char op;
    cout << "Enter first number: ";
    cin >> num1;
    cout << "Enter second number: ";
    cin >> num2;
    cout << "Enter operator (+, -, *, /, %): ";
    cin >> op;
    switch (op)
    {
        case '+':
            cout << "Addition = " << num1 + num2;
            break;
        case '-':
            cout << "Subtraction = " << num1 - num2;
            break;
        case '*':
            cout << "Multiplication = " << num1 * num2;
            break;
        case '/':
            if (num2 != 0)
```

```
                cout << "Division = " << (float)num1 / num2;
            else
                cout << "Division by zero is not allowed.";
            break;
        case '%':
            if (num2 != 0)
                cout << "Modulus = " << num1 % num2;
            else
                cout << "Modulus by zero is not allowed.";
            break;
        default:
            cout << "Invalid Operator!";
    }
    return 0;
}
```

Output 1:

```
Enter first number: 20
Enter second number: 10
Enter operator (+, -, *, /, %): +
Addition = 30
```

Output 2:

```
Enter first number: 25
Enter second number: 5
Enter operator (+, -, *, /, %): /
Division = 5
```

Output 3:

```
Enter first number: 15
Enter second number: 4
Enter operator (+, -, *, /, %): %
Modulus = 3
```

Code Explanation:

The program reads two numbers and an operator from the user. A ‘switch‘ statement selects the matching arithmetic operation (‘+‘, ‘-‘, ‘*‘, ‘/‘ or ‘%‘). Before division or modulus, it checks that the second number is not zero; an unsupported operator reaches the default case. The selected result or an appropriate error message is printed.

Conclusion:

The calculator performed the selected arithmetic operation using a `switch-case` statement. The sample operations produced the expected results, and the program included checks for division or modulus by zero and an invalid operator. This practical demonstrated multi-way selection and arithmetic operators.

Program - 3

Aim:

To write a C++ program to convert a given number of days into months and remaining days.

Theory:

This program uses the assumption that one month contains 30 days. Integer division gives the number of complete months, while the modulus operator gives the remaining days. The calculations are `months = days / 30` and `remainingDays = days % 30`. For example, 75 days corresponds to 2 complete months and 15 remaining days. This is an approximate conversion because calendar months do not all contain 30 days.

Integer division deliberately discards the fractional part because only complete 30-day months are counted. The remainder is always less than 30 for a non-negative input, which makes it the suitable value for the remaining days. The result should be interpreted as an estimate rather than a calendar-date conversion.

Algorithm:

Step - 1 Start the program.
Step - 2 Declare integer variables for the input number of days, complete months and remaining days.
Step - 3 Prompt the user and read the total number of days.
Step - 4 Assume each month contains 30 days and calculate complete months using integer division: `months = days / 30`.
Step - 5 Calculate the leftover days using the remainder operation: `remainingDays = days % 30`.
Step - 6 Display both the complete-month count and the remaining-day count with meaningful labels.
Step - 7 Stop the program. Note that this is an approximate conversion based on 30-day months.

Code:

```cpp
#include <iostream>
using namespace std;
int main()
{
    int days, months, remainingDays;
    cout << "Enter the number of days: ";
    cin >> days;
    months = days / 30;
    remainingDays = days % 30;
    cout << "\nEquivalent Time:";
    cout << "\nMonths = " << months;
    cout << "\nRemaining Days = " << remainingDays;
    return 0;
}
```

Output 1:

```
Enter the number of days: 75

Equivalent Time:
Months = 2
Remaining Days = 15
```

Output 2:

```
Enter the number of days: 120

Equivalent Time:
Months = 4
Remaining Days = 0
```

Output 3:

```
Enter the number of days: 49

Equivalent Time:
Months = 1
Remaining Days = 19
```

Code Explanation:

The program reads a number of days and converts it using the assumption that one month contains 30 days. Integer division (‘days / 30‘) gives the number of complete months, while the modulus operator (‘days % 30‘) gives the remaining days. Both values are then displayed with labels.

Conclusion:

The program converted the entered number of days into complete 30-day months and remaining days using integer division and the modulus operator. The sample inputs produced the expected quotient and remainder. This practical demonstrated integer arithmetic and the importance of stating the 30-day-month assumption.

Program - 4

Aim:

To write a C++ program to enter the marks of six subjects and find the total marks and percentage.

Theory:

The marks of six subjects are stored in variables and added to obtain the total. If each subject is out of 100, the maximum total is 600 and the percentage is calculated as `(total / 600.0) * 100`, equivalently `total / 6.0`. Using a floating-point constant such as `6.0` ensures that fractional results are retained instead of being truncated by integer division. The program therefore demonstrates arithmetic expressions, variable selection and formatted output.

For six subjects carrying equal maximum marks of 100 each, the maximum total is 600. Dividing the total by 6.0 gives the average marks out of 100, numerically equal to the percentage under this assumption. The formula must be adjusted if the subjects have different maximum marks.

Algorithm:

Step - 1 Start the program and declare variables to store the marks of all six subjects, their total and the percentage.
Step - 2 Prompt for the marks of each subject and read all six values.
Step - 3 Add the six subject marks and store the sum in `total`.
Step - 4 Calculate the percentage using the total out of 600; use floating-point arithmetic, for example `total / 6.0`, to avoid unintended integer division.
Step - 5 Display the total marks out of 600 and the calculated percentage.
Step - 6 Stop the program.

Code:

```cpp
#include <iostream>
using namespace std;
int main()
{
    int sub1, sub2, sub3, sub4, sub5, sub6;
    int total;
    float percentage;
    cout << "Enter marks of Subject 1: ";
    cin >> sub1;
    cout << "Enter marks of Subject 2: ";
    cin >> sub2;
    cout << "Enter marks of Subject 3: ";
    cin >> sub3;
    cout << "Enter marks of Subject 4: ";
    cin >> sub4;
    cout << "Enter marks of Subject 5: ";
    cin >> sub5;
    cout << "Enter marks of Subject 6: ";
    cin >> sub6;
    total = sub1 + sub2 + sub3 + sub4 + sub5 + sub6;
    percentage = total / 6.0;
    cout << "\nTotal Marks = " << total;
    cout << "\nPercentage = " << percentage << "%";
    return 0;
}
```

Output:

```
Enter marks of Subject 1: 85
Enter marks of Subject 2: 78
Enter marks of Subject 3: 90
Enter marks of Subject 4: 88
Enter marks of Subject 5: 76
Enter marks of Subject 6: 83

Total Marks = 500
Percentage = 83.3333%
```

Code Explanation:

The program accepts marks for six subjects and adds them to a running total. It calculates the percentage by dividing the total by the maximum possible marks and multiplying by 100. A floating-point calculation is used so that fractional percentages are not lost. The total and percentage are printed at the end.

Conclusion:

The program calculated the total marks and percentage for six subjects. For the sample marks, the total was 500 and the percentage was approximately 83.33%. Using floating-point division preserved the fractional part of the percentage. This practical demonstrated arithmetic expressions and appropriate data-type selection.

Program - 5

Aim:

To write a C++ program to find whether a given number is even or odd.

Theory:

An integer is even when it is divisible by 2 without a remainder; otherwise, it is odd. The modulus operator `%` returns the remainder after division, so the condition `num % 2 == 0` identifies an even number. The `if-else` statement executes one of two alternatives depending on whether the condition is true or false. The equality operator `==` compares values, while `=` assigns a value to a variable.

The remainder test works for positive and negative integers because an integer divisible by 2 has no remainder. The conditional statement selects exactly one of the two messages, making this a simple example of decision-making in a program.

Algorithm:

Step - 1 Start the program and declare an integer variable `num`.
Step - 2 Prompt the user and read the integer to be tested.
Step - 3 Calculate the remainder when the number is divided by 2 using `num % 2`.
Step - 4 If the remainder equals zero, select the even-number message.
Step - 5 Otherwise, select the odd-number message because the number is not divisible by 2 without a remainder.
Step - 6 Display the appropriate message and stop.

Code:

```cpp
#include <iostream>
using namespace std;
int main()
{
    int num;
    cout << "Enter a number: ";
    cin >> num;
    if (num % 2 == 0)
        cout << num << " is an Even number.";
    else
        cout << num << " is an Odd number.";
    return 0;
}
```

Output 1:

```
Enter a number: 12
12 is an Even number.
```

Output 2:

```
Enter a number: 7
7 is an Odd number.
```

Code Explanation:

The program reads an integer and evaluates ‘num % 2‘. A remainder of zero means the number is even; otherwise it is odd. An ‘if-else‘ statement chooses the appropriate message, demonstrating a simple conditional decision.

Conclusion:

The program identified whether an entered integer was even or odd by checking its remainder when divided by 2. The sample values were classified correctly. This practical demonstrated the modulus operator, comparison using `==`, and decision-making with `if-else`.

Program - 6

Aim:

To write a C++ program to find the largest among three numbers.

Theory:

The largest of three values can be found by comparing each value with the other two. Relational operators such as `>=` produce a true or false result, and the logical AND operator `&&` combines two conditions. An `if-else if-else` ladder checks the conditions in order and executes the first matching branch. Using `>=` also handles cases in which two or more values are equal.

The comparisons cover all possible relationships among the three values. Using greater-than-or-equal comparisons means that the program still identifies a valid maximum when two or more inputs are equal. A sequence of if/else-if/else branches ensures that only one result branch is selected.

Algorithm:

Step - 1 Start the program and declare integer variables `a`, `b` and `c`.
Step - 2 Read all three values from the user.
Step - 3 Compare `a` with both `b` and `c`; if `a >= b` and `a >= c`, select `a` as the largest.
Step - 4 If the first condition is false, compare `b` with `a` and `c`; if both comparisons hold, select `b`.
Step - 5 If neither of the first two conditions holds, select `c` as the largest.
Step - 6 Display the selected largest value. The use of `>=` also handles equal maximum values.
Step - 7 Stop the program.

Code:

```cpp
#include <iostream>
using namespace std;
int main()
{
    int a, b, c;
    cout << "Enter three numbers: ";
    cin >> a >> b >> c;
    if (a >= b && a >= c)
        cout << "Largest number is: " << a;
    else if (b >= a && b >= c)
        cout << "Largest number is: " << b;
    else
        cout << "Largest number is: " << c;
    return 0;
}
```

Output:

```
Enter three numbers: 10 25 15
Largest number is: 25
```

Code Explanation:

The program reads three numbers and compares them using relational operators. The ‘if‘, ‘else if‘ and ‘else‘ branches identify which value is greatest, including cases where the first or second number is the largest. The selected maximum is displayed.

Conclusion:

The program determined the largest of three entered numbers by comparing each value with the other two. For the sample input 10, 25 and 15, it correctly displayed 25. This practical demonstrated relational operators, logical AND, and the use of an `if-else if-else` ladder.

Program - 7

Aim:

To write a C++ program to check whether a given number is an Armstrong number or not.

Theory:

An Armstrong number is equal to the sum of its digits, each raised to the power given by the number of digits. For example, 371 is an Armstrong number because $3^3 + 7^3 + 1^3 = 371$. The modulus operator `% 10` extracts the last digit, and integer division by 10 removes that digit. A loop counts the digits and another loop calculates the powered sum. The `pow()` function from `<cmath>` performs exponentiation; rounding its result helps avoid floating-point representation issues before adding it to an integer total.

The number of digits must be determined before the digit powers are accumulated, because every digit is raised to that same count. A working copy is used for digit extraction so that the original input remains available for the final equality test. This common definition is most often applied to non-negative integers.

Algorithm:

Step - 1 Start the program and read the integer `num`; preserve its original value for the final comparison.

Step - 2 Count the digits by repeatedly dividing a temporary copy of the number by 10 until no digits remain; handle zero as a one-digit number if required.

Step - 3 Initialise `result` to zero and restore a working copy of the original number.

Step - 4 Extract the last digit with `temp % 10` and calculate that digit raised to the number of digits.

Step - 5 Add the powered digit to `result`, then remove the last digit using integer division by 10; repeat until all digits are processed.

Step - 6 Compare the computed sum with the original number and display whether it is an Armstrong number.

Step - 7 Stop the program.

Code:

```cpp
#include <iostream>
#include <cmath>
using namespace std;
int main()
{
    int num, originalNum, remainder, digits = 0;
    int result = 0;
    cout << "Enter a number: ";
    cin >> num;
    originalNum = num;
    // Count the number of digits
    for (; originalNum != 0; originalNum /= 10)
        digits++;
    originalNum = num;
    // Sum of digits raised to the power of number of digits
    for (; originalNum != 0; originalNum /= 10)
    {
        remainder = originalNum % 10;
        result += (int)round(pow(remainder, digits));
    }
    if (result == num)
        cout << num << " is an Armstrong Number.";
```

```
    else
        cout << num << " is not an Armstrong Number.";
    return 0;
}
```

Output 1:

```
Enter a number: 371
371 is an Armstrong Number.
```

Output 2:

```
Enter a number: 123
123 is not an Armstrong Number.
```

Code Explanation:

The program first counts the digits in the input number. It then extracts each digit using '% 10', raises the digit to the power of the digit count with 'pow()', and adds the result to a sum. Finally, it compares this sum with the original number to decide whether it is an Armstrong number. The original value is preserved for the comparison.

Conclusion:

The program checked the Armstrong-number condition by counting the digits and summing each digit raised to that count. The sample number 371 was identified as an Armstrong number, whereas 123 was not. This practical demonstrated loops, digit extraction, exponentiation and comparison of the calculated sum with the original number.

Program - 8

Aim:

To write a C++ program to generate the multiplication table of a given number.

Theory:

A `for` loop repeats a statement block while its condition remains true. It contains an initialisation, a condition and an update expression. In this program, the loop variable runs from 1 through 10, and each iteration prints the product of the input number and the current loop value. This avoids writing a separate output statement for every row of the multiplication table.

The loop repeats a fixed number of times, so it is appropriate for producing the first ten multiples. The counter is updated after each iteration, and the loop ends once it exceeds 10. This demonstrates how initialisation, condition checking and updating work together in a for loop.

Algorithm:

Step - 1 Start the program and declare integer variables `num` and `i`.
Step - 2 Read the number whose multiplication table is required.
Step - 3 Initialise the loop counter `i` to 1.
Step - 4 While `i <= 10`, calculate `num * i` and display the expression and product.
Step - 5 Increase `i` by one after each iteration and repeat until the tenth multiple has been displayed.
Step - 6 Stop the program.

Code:

```cpp
#include <iostream>
using namespace std;
int main()
{
    int num;
    cout << "Enter a number: ";
    cin >> num;
    cout << "\nMultiplication Table of " << num << ":\n";
    for (int i = 1; i <= 10; i++)
        cout << num << " x " << i << " = " << num * i << endl;
    return 0;
}
```

Output:

```
Enter a number: 7

Multiplication Table of 7:
7 x 1 = 7
7 x 2 = 14
7 x 3 = 21
7 x 4 = 28
7 x 5 = 35
7 x 6 = 42
7 x 7 = 49
7 x 8 = 56
7 x 9 = 63
7 x 10 = 70
```

Code Explanation:

The program reads the number whose table is required and starts a ‘for‘ loop at 1. The loop continues through 10, calculating ‘num * i‘ on each iteration. Each product is printed as one line, and the loop stops after the tenth multiple.

Conclusion:

The program generated the multiplication table of the entered number from 1 through 10. The sample input 7 produced the expected products. This practical demonstrated how a `for` loop repeats a calculation and formats repeated output.

Program - 9

Aim:

To write a C++ program that demonstrates the concept of a reference variable.

Theory:

A reference variable is an alternative name for an existing variable. It is declared with the reference operator, for example `int &ref = num;`, and must be initialised when it is declared. The reference and the original variable refer to the same object, so modifying the value through either name changes the same stored value. References are commonly used to pass arguments to functions when changes to the caller's variables are required.

Unlike an ordinary copied variable, a reference does not hold an independent value; it is another name for the same memory location. Therefore, assignment through the reference is visible when the original variable is read. A reference must be initialised at declaration and cannot later be redirected to another variable.

Algorithm:

Step - 1 Start the program and declare an integer variable `num`.
Step - 2 Read the initial value of `num` from the user.
Step - 3 Declare `int &ref = num`; this binds `ref` as an alias for the existing variable rather than creating a separate integer.
Step - 4 Display both `num` and `ref` to show that they initially refer to the same stored value.
Step - 5 Increase the value through the reference, for example `ref = ref + 10`.
Step - 6 Display both names again and observe that the original variable has changed as well.
Step - 7 Stop the program.

Code:

```cpp
#include <iostream>
using namespace std;
int main()
{
    int num;
    cout << "Enter a number: ";
    cin >> num;
    int &ref = num;   // Reference variable
    cout << "\nOriginal value of num = " << num << endl;
    cout << "Value using reference variable = " << ref << endl;
    ref = ref + 10;   // Modify through reference
    cout << "\nAfter modifying through reference variable:" << endl;
    cout << "Value of num = " << num << endl;
    cout << "Value of ref = " << ref << endl;
    return 0;
}
```

Output:

```
Enter a number: 25

Original value of num = 25
Value using reference variable = 25

After modifying through reference variable:
Value of num = 35
Value of ref = 35
```

Code Explanation:

The program declares a reference variable that acts as another name for an existing variable. After initialisation, using the reference to read or change a value accesses the original variable directly; it does not create a separate copy. The displayed values demonstrate that updates through the reference are reflected in the original.

Conclusion:

The program demonstrated that a reference variable is another name for an existing variable. Changing the value through the reference also changed the original variable. This practical illustrated reference declaration, aliasing and the use of references to access the same object.

Program - 10

Aim:

To write a C++ program to swap two numbers using the call by value method.

Theory:

In call by value, a function receives copies of the values passed by the caller. The function can modify its local parameters, but those changes do not affect the original variables. In the swapping example, a temporary variable is used to exchange the copied values. The swapped values can be observed inside the function, while the variables in `main()` retain their original values after the call.

The function parameters have their own local storage and initially receive the caller's values. The temporary variable prevents the first value from being lost during the exchange. Since the parameters are copies, the demonstration distinguishes local modification from modification of the caller's variables.

Algorithm:

Step - 1 Start the program and read the two numbers `x` and `y` in `main()`.
Step - 2 Display the original values before calling the swapping function.
Step - 3 Call `swapValues(x, y)`; the function receives copies of the two arguments because the parameters are passed by value.
Step - 4 Inside the function, store the first local parameter in a temporary variable, assign the second parameter to the first, and assign the temporary value to the second.
Step - 5 Return from the function and display `x` and `y` again in `main()`.
Step - 6 Observe that the caller's original variables remain unchanged because only local copies were swapped; stop.

Code:

```cpp
#include <iostream>
using namespace std;
// Function using Call by Value
void swapValues(int a, int b)
{
    int temp;
    temp = a;
    a = b;
    b = temp;
    cout << "\nInside swapValues() function:" << endl;
    cout << "a = " << a << ", b = " << b << endl;
}
int main()
{
    int x, y;
    cout << "Enter two numbers: ";
    cin >> x >> y;
    cout << "\nBefore swapping:" << endl;
    cout << "x = " << x << ", y = " << y << endl;
    swapValues(x, y);
    cout << "\nAfter calling swapValues() function:" << endl;
    cout << "x = " << x << ", y = " << y << endl;
    return 0;
}
```

Output:

```
Enter two numbers: 10 20

Before swapping:
x = 10, y = 20

Inside swapValues() function:
a = 20, b = 10

After calling swapValues() function:
x = 10, y = 20
```

Code Explanation:

The program passes two numbers to a swapping function by value. The function receives local copies, swaps those copies with a temporary variable, and displays or uses the swapped local values. Since the original variables are not passed by reference, their values in ‘main()‘ remain unchanged after the function returns.

Conclusion:

The program swapped the parameter values inside the function, but the original variables remained unchanged after the call. This confirmed that call by value passes copies of arguments. The practical highlighted the difference between modifying local parameters and modifying the caller’s variables.

Program - 11

Aim:

To write a C++ program to swap two numbers using the call by reference method.

Theory:

In call by reference, function parameters are references to the caller's variables rather than independent copies. Parameters such as `int &a` and `int &b` allow the function to work directly with the original arguments. Swapping the two parameters therefore changes the variables in the calling function. This method is useful when a function must modify its arguments or avoid copying larger objects.

Reference parameters provide direct access to the variables supplied by the caller, so assignments inside the function persist after it returns. A temporary variable is still needed to preserve one value during the exchange. This method is useful when a function is intended to modify more than one original variable.

Algorithm:

Step - 1 Start the program and read the two integer values `num1` and `num2`.
Step - 2 Display both original values before the function call.
Step - 3 Call `swapNumbers(num1, num2)` with reference parameters so the function aliases the caller's variables.
Step - 4 Inside the function, save the first value in a temporary variable, copy the second value into the first, and then place the saved value into the second.
Step - 5 Return to `main()` and display the values after the call.
Step - 6 Confirm that the original variables have exchanged values, then stop.

Code:

```
#include <iostream>
using namespace std;
// Function using Call by Reference
void swapNumbers(int &a, int &b)
{
    int temp;
    temp = a;
    a = b;
    b = temp;
}
int main()
{
    int num1, num2;
    cout << "Enter first number: ";
    cin >> num1;
    cout << "Enter second number: ";
    cin >> num2;
    cout << "\nBefore Swapping:" << endl;
    cout << "First Number = " << num1 << endl;
    cout << "Second Number = " << num2 << endl;
    swapNumbers(num1, num2);
    cout << "\nAfter Swapping:" << endl;
    cout << "First Number = " << num1 << endl;
    cout << "Second Number = " << num2 << endl;
    return 0;
}
```

Output:

```
Enter first number: 25
Enter second number: 40

Before Swapping:
First Number = 25
Second Number = 40

After Swapping:
First Number = 40
Second Number = 25
```

Code Explanation:

The swapping function receives its parameters by reference, so each parameter refers directly to the corresponding variable in ‘main()‘. A temporary variable is used to exchange the two values. Because the function operates on the originals, the swapped values remain changed after the function returns.

Conclusion:

The program successfully swapped two numbers using reference parameters. Since the function operated on the original variables, the exchanged values remained visible after the function returned. This practical demonstrated call by reference and how references allow a function to modify its arguments.

Program - 12

Aim:

To write a C++ program that uses a function with default arguments.

Theory:

A default argument supplies a value for a function parameter when the caller omits that argument. For example, a function declared as `add(int a = 10, int b = 20)` can be called with zero, one or two arguments. If an argument is supplied, it replaces the corresponding default; otherwise, the default value is used. Default arguments must be specified from right to left, so parameters following a parameter with a default must also have defaults.

Default arguments are specified in the function declaration and are applied by the compiler when trailing arguments are omitted. Arguments supplied by the caller always take precedence over the defaults. This feature can simplify function calls when a parameter commonly uses the same value.

Algorithm:

Step - 1 Start the program and define `add()` with default values for its parameters, such as `a = 10` and `b = 20`.
Step - 2 Make a call with no arguments; both parameters take their declared default values.
Step - 3 Make a call with one argument; the supplied value is used for the first parameter and the second parameter keeps its default.
Step - 4 Make a call with two arguments; both default values are replaced by the supplied arguments.
Step - 5 For each call, calculate and display the returned sum so the effect of omitted arguments can be compared.
Step - 6 Stop the program.

Code:

```cpp
#include <iostream>
using namespace std;
// Function with default arguments
int add(int a = 10, int b = 20)
{
    return a + b;
}
int main()
{
    cout << "add()       = " << add() << endl;       // Uses both defaults
    cout << "add(5)      = " << add(5) << endl;      // a = 5, b = 20
    cout << "add(5, 15) = " << add(5, 15) << endl; // a = 5, b = 15
    return 0;
}
```

Output:

```
add()       = 30
add(5)      = 25
add(5, 15) = 20
```

Code Explanation:

The function declaration supplies default values for parameters. When the function is called without arguments, both defaults are used; when one argument is supplied, the other parameter keeps its default; when both are supplied, the passed values replace the defaults. The calls and printed results demonstrate how default arguments reduce repeated function definitions.

Conclusion:

The function produced results using both default parameter values, one supplied argument, and two supplied arguments. The outputs demonstrated that omitted arguments use their declared defaults. This practical showed how default arguments make a function flexible without requiring separate overloads for each call.

Program - 13

Aim:

To write a C++ program to calculate the factorial of a number using recursion.

Theory:

Recursion occurs when a function calls itself to solve a smaller instance of a problem. A recursive function needs a base case to stop further calls and a recursive case that moves towards that base case. The factorial is defined by $n! = n(n-1)!$, with $0! = 1$ and $1! = 1$. The function returns 1 for the base cases and otherwise returns `n * factorial(n - 1)`. Factorial is defined for non-negative integers, and its value grows quickly, so the range of the chosen integer type must be considered.

Each recursive call creates a smaller factorial problem until the base case is reached; the pending multiplications are then evaluated as the calls return. For example, 4! is calculated as $4 \times 3 \times 2 \times 1$. Factorials grow quickly, so the chosen integer type limits the largest value that can be represented safely.

Algorithm:

Step - 1 Start the program and define the recursive function `factorial(n)`.

Step - 2 Implement the base case: return 1 when `n` is 0 or 1, which prevents recursion from continuing indefinitely.

Step - 3 For other non-negative values, return `n * factorial(n-1)` so each call reduces the argument toward the base case.

Step - 4 Read the number in `main()` and check whether it is negative.

Step - 5 If the input is negative, display that factorial is not defined for it; otherwise call `factorial(num)`.

Step - 6 Display the returned factorial value and stop.

Code:

```cpp
#include <iostream>
using namespace std;
// Recursive function to calculate factorial
int factorial(int n)
{
    if (n == 0 || n == 1)
        return 1;
    else
        return n * factorial(n - 1);
}
int main()
{
    int num;
    cout << "Enter a number: ";
    cin >> num;
    if (num < 0)
        cout << "Factorial of a negative number is not defined.";
    else
        cout << "Factorial of " << num << " = " << factorial(num);
    return 0;
}
```

Output:

```
Enter a number: 5
Factorial of 5 = 120
```

Code Explanation:

The recursive ‘factorial()‘ function returns 1 for the base cases 0 and 1. For any larger positive input, it returns ‘n * factorial(n - 1)‘, repeatedly reducing the problem until a base case is reached. The result is then returned through the waiting calls. The main function reads the input, handles negative values and prints the factorial for valid input.

Conclusion:

The recursive function calculated the factorial of the sample input 5 as 120. Its base case stopped the recursive calls, while the recursive case reduced the argument towards that base case. This practical demonstrated recursive problem-solving and the importance of handling negative input.

Program - 14

Aim:

To write a C++ program to take input in an array and print the sum of its elements.

Theory:

An array stores a fixed number of elements of the same data type in consecutive memory locations. Each element is accessed using an index, beginning at 0. A loop can read the values and another loop can process them. In this program, a running total is initialised to zero and each array element is added to it. The number of elements entered must not exceed the declared array capacity.

The array's valid indices begin at zero and end at $n - 1$, so both loops must use the same index range. Initialising the sum to zero is essential because each element is accumulated into it. The number of input elements must remain within the declared array capacity.

Algorithm:

Step - 1 Start the program and declare an array `arr[100]`, an element count `n`, and initialise `sum` to zero.
Step - 2 Read the number of elements and ensure it does not exceed the array capacity.
Step - 3 Use a loop from index 0 to `n-1` to read each value into the array.
Step - 4 Run a second loop over the same valid indices and add each array element to `sum`.
Step - 5 After all elements have been processed, display the final sum.
Step - 6 Stop the program.

Code:

```cpp
#include <iostream>
using namespace std;
int main()
{
    int arr[100], n, sum = 0;
    cout << "Enter the number of elements: ";
    cin >> n;
    cout << "Enter " << n << " elements:" << endl;
    for (int i = 0; i < n; i++)
        cin >> arr[i];
    // Calculate sum of array elements
    for (int i = 0; i < n; i++)
        sum = sum + arr[i];
    cout << "Sum of array elements = " << sum << endl;
    return 0;
}
```

Output:

```
Enter the number of elements: 5
Enter 5 elements:
10
20
30
40
50
Sum of array elements = 150
```

Code Explanation:

The program stores the input values in an array and uses a loop to visit each element. Each element is added to a sum variable, which is initialised to zero before the loop. Once all elements have been processed, the total is displayed. The loop counter ensures that every valid array position is visited once.

Conclusion:

The program read the array elements and added them to obtain the total. For the sample values 10, 20, 30, 40 and 50, the sum was 150. This practical demonstrated array indexing, loop-based traversal and accumulation of values.

Program - 15

Aim:

To write a C++ program to read a string and print the total number of characters.

Theory:

A C++ `string` stores a sequence of characters and provides member functions for common text operations. The function `getline(cin, str)` reads a complete line, including spaces, while `str.length()` returns the number of characters in the string. Spaces and punctuation are included in this count. This program demonstrates how the standard string class simplifies text input and length calculation.

A string's length is the number of characters stored in it, including spaces and punctuation but not the terminating null character used by C-style strings. An empty line has length zero. The string member function avoids the need to manually inspect each character with a loop.

Algorithm:

Step - 1 Start the program and declare a string variable `str`.
Step - 2 Prompt the user and read a complete line using `getline(cin, str)` so spaces are included.
Step - 3 Call `str.length()` to obtain the number of characters stored in the string.
Step - 4 Store or directly use the returned length and display it with a descriptive message.
Step - 5 Stop the program.

Code:

```cpp
#include <iostream>
#include <string>
using namespace std;
int main()
{
    string str;
    cout << "Enter a string: ";
    getline(cin, str);
    cout << "Total number of characters = " << str.length() << endl;
    return 0;
}
```

Output:

```
Enter a string: Hello World
Total number of characters = 11
```

Code Explanation:

The program reads a line of text and examines its characters. A loop or the string length operation is used to determine how many characters are present, including spaces when the whole line is read with 'getline()'. The final count is displayed. This demonstrates string input and basic string processing.

Conclusion:

The program read a complete line and used the string member function `length()` to count its characters. The text `Hello World` produced a length of 11, including the space. This practical demonstrated basic string input and a standard string member function.

Program - 16

Aim:

To write a C++ program to create a class `Student` with Name, ID, CPI, SPI and Semester as data members, and member functions to read and display the information.

Theory:

A class is a user-defined type that combines data members and member functions in one unit, while an object is an instance of that class. Encapsulation groups data with the operations that use it. Access specifiers such as `private` restrict direct access to class members, whereas `public` members form the interface available to the rest of the program. The `Student` class stores student details and provides member functions to accept and display them.

The object
texttts contains its own student data, and its member functions operate on that object's state. Keeping data and related operations together illustrates encapsulation and makes the class reusable for creating other student objects. The input and display functions provide a simple interface to the stored information.

Algorithm:

Step - 1 Start the program and define the `Student` class with members for name, ID, CPI, SPI and semester.
Step - 2 Place the data members and member functions under the appropriate access specifiers to keep the class organised.
Step - 3 Implement `getData()` to prompt for and store each student's detail.
Step - 4 Implement `displayData()` to print the stored values with clear labels.
Step - 5 In `main()`, create a `Student` object named `s`.
Step - 6 Call `s.getData()` to initialise the object's information, then call `s.displayData()` to show it.
Step - 7 Stop the program.

Code:

```
#include <iostream>
#include <string>
using namespace std;
class Student
{
private:
    string name;
    int id;
    float cpi, spi;
    int semester;
public:
    // Member function to accept student details
    void getData()
    {
        cout << "Enter Student Name: ";
        getline(cin, name);
        cout << "Enter Student ID: ";
        cin >> id;
        cout << "Enter CPI: ";
        cin >> cpi;
        cout << "Enter SPI: ";
        cin >> spi;
        cout << "Enter Semester: ";
        cin >> semester;
```

```
    }
    // Member function to display student details
    void displayData()
    {
        cout << "\n----- Student Information -----" << endl;
        cout << "Name     : " << name << endl;
        cout << "ID       : " << id << endl;
        cout << "CPI      : " << cpi << endl;
        cout << "SPI      : " << spi << endl;
        cout << "Semester : " << semester << endl;
    }
};
int main()
{
    Student s;
    s.getData();
    s.displayData();
    return 0;
}
```

Output:

```
Enter Student Name: Rahul Patel
Enter Student ID: 101
Enter CPI: 8.75
Enter SPI: 9.10
Enter Semester: 5

----- Student Information -----
Name     : Rahul Patel
ID       : 101
CPI      : 8.75
SPI      : 9.1
Semester : 5
```

Code Explanation:

The program defines a ‘Student‘ class with data members for student information and member functions to assign or display those values. In ‘main()‘, an object of the class is created and its member functions are called. The object keeps related data together, while the class describes the structure and behaviour shared by its objects.

Conclusion:

The `Student` class stored student information and used member functions to accept and display the details. The practical demonstrated the relationship between classes and objects, encapsulation, access specifiers and member-function calls.

Program - 17

Aim:

To write a C++ program to count the number of objects created using a static variable.

Theory:

A static data member belongs to the class rather than to an individual object. Only one shared copy exists, and all objects of the class access the same member. It is useful for maintaining class-wide information, such as the number of objects created. In this program, the constructor updates the shared counter whenever an object is created, allowing the count to be displayed for multiple objects.

A static data member is defined once for the class and shared by all its instances, rather than duplicated in every object. The constructor provides a convenient point to update the count whenever an object is created. A static member function can be called without creating an additional object and can access static members directly.

Algorithm:

Step - 1 Start the program and declare the class's static data member `count`; define and initialise it outside the class to zero.
Step - 2 In the constructor, increment the shared `count` whenever a new object is constructed.
Step - 3 Display the current count from the constructor if the program is designed to show each object's creation number.
Step - 4 Define the static member function `displayCount()` to read and print the class-wide count.
Step - 5 Create three objects in `main()`, allowing the constructor to update the shared count for each one.
Step - 6 Call `Student::displayCount()` using the class name and scope-resolution operator.
Step - 7 Stop the program.

Code:

```
#include <iostream>
using namespace std;
class Student
{
private:
    static int count;   // Static data member
public:
    // Constructor
    Student()
    {
        count++;
        cout << "Object " << count << " created." << endl;
    }
    // Static member function
    static void displayCount()
    {
        cout << "\nTotal number of objects created = " << count << endl;
    }
};
// Definition of static data member
int Student::count = 0;
int main()
{
    Student s1;
    Student s2;
    Student s3;
    Student::displayCount();
```

```
28      return 0;
29  }
```

Output:

```
1  Object 1 created.
2  Object 2 created.
3  Object 3 created.
4
5  Total number of objects created = 3
```

Code Explanation:

The class contains a static data member that is shared by all objects rather than stored separately in each object. The constructor increments this shared counter whenever a new object is created. The program creates several objects and displays the counter, demonstrating how static data tracks class-wide information.

Conclusion:

The program used a static data member to maintain a shared count across objects of the class. The count changed as objects were created, demonstrating that a static member is shared rather than separately stored in each object. This practical illustrated class-level data and object creation.

Program - 18

Aim:

To write a C++ program for a bank account class with the data members name of depositor, account number, withdrawal amount and balance, and the member functions to assign initial values, deposit an amount, withdraw an amount after checking the balance, and display the name and balance, and to handle n account holders using an array of objects.

Theory:

An array of objects stores several instances of the same class and allows each object to maintain its own data. A bank-account class can group details such as the account holder's name, account number and balance with member functions for deposits, withdrawals and displaying information. Deposits increase the balance, while withdrawals should be checked against the available balance before being accepted. The array allows the program to process multiple accounts using loops.

Each element of the array is a separate account object, so each account maintains its own balance and identifying details. Deposits and withdrawals update the balance stored in that object, not the other accounts. The balance check prevents a withdrawal larger than the available funds; practical banking software would also validate negative amounts and use suitable monetary precision.

Algorithm:

Step - 1 Start the program and define the `BankAccount` class with data for account holder, account number, balance and withdrawal amount.

Step - 2 Implement `assign()` to read the account holder's details and initialise the account balance.

Step - 3 Implement `deposit()` to read a deposit amount and add it to the account's balance.

Step - 4 Implement `withdraw()` to read the requested withdrawal and compare it with the available balance before deducting it.

Step - 5 Implement `display()` to show the account holder, account number and updated balance.

Step - 6 Read the number of account holders and create an array containing that many `BankAccount` objects.

Step - 7 For each object, assign its initial details, process the deposit and withdrawal, and display the final account information.

Step - 8 Stop the program.

Code:

```
#include <iostream>
#include <string>
using namespace std;
class BankAccount
{
private:
    string name;
    int accountNo;
    float balance;
    float withdrawAmount;
public:
    // Assign initial values
    void assign()
    {
        cout << "Enter Depositor Name: ";
        cin.ignore();
        getline(cin, name);
        cout << "Enter Account Number: ";
        cin >> accountNo;
```

```
        cout << "Enter Initial Balance: ";
        cin >> balance;
    }
    // Deposit an amount
    void deposit()
    {
        float amount;
        cout << "Enter amount to deposit: ";
        cin >> amount;
        balance = balance + amount;
        cout << "Amount Deposited Successfully." << endl;
    }
    // Withdraw an amount after checking balance
    void withdraw()
    {
        cout << "Enter amount to withdraw: ";
        cin >> withdrawAmount;
        if (withdrawAmount <= balance)
        {
            balance = balance - withdrawAmount;
            cout << "Withdrawal Successful." << endl;
        }
        else
            cout << "Insufficient Balance!" << endl;
    }
    // Display name and balance
    void display()
    {
        cout << "\n-----------------------------" << endl;
        cout << "Depositor Name : " << name << endl;
        cout << "Account Number : " << accountNo << endl;
        cout << "Balance        : " << balance << endl;
    }
};
int main()
{
    int n;
    cout << "Enter Number of Account Holders: ";
    cin >> n;
    BankAccount *acc = new BankAccount[n];   // Array of objects
    for (int i = 0; i < n; i++)
    {
        cout << "\nEnter Details of Account Holder " << i + 1 << endl;
        acc[i].assign();
    }
    for (int i = 0; i < n; i++)
    {
        cout << "\nTransactions for Account Holder " << i + 1 << endl;
        acc[i].deposit();
        acc[i].withdraw();
    }
    cout << "\n\n----- Account Details -----";
    for (int i = 0; i < n; i++)
        acc[i].display();
    delete[] acc;
    return 0;
}
```

Output:

```
Enter Number of Account Holders: 2

Enter Details of Account Holder 1
Enter Depositor Name: Rahul Patel
Enter Account Number: 1001
Enter Initial Balance: 5000

Enter Details of Account Holder 2
Enter Depositor Name: Neha Shah
Enter Account Number: 1002
Enter Initial Balance: 8000

Transactions for Account Holder 1
Enter amount to deposit: 1000
Amount Deposited Successfully.
Enter amount to withdraw: 3000
Withdrawal Successful.

Transactions for Account Holder 2
Enter amount to deposit: 2000
Amount Deposited Successfully.
Enter amount to withdraw: 12000
Insufficient Balance!


----- Account Details -----
-----------------------------
Depositor Name : Rahul Patel
Account Number : 1001
Balance        : 3000

-----------------------------
Depositor Name : Neha Shah
Account Number : 1002
Balance        : 10000
```

Code Explanation:

The program defines a ‘BankAccount‘ class with account details and functions for assigning data, depositing, withdrawing and displaying the balance. An array of objects stores multiple account holders. Loops process each account, and the withdrawal function checks whether the requested amount is within the available balance before subtracting it. The dynamically allocated array is released with ‘delete[]‘ at the end.

Conclusion:

The program used an array of bank-account objects to organise account information and perform the operations supported by the class. The practical demonstrated how objects keep related data and member functions together and how arrays and loops can be used to process multiple accounts. Any withdrawal should be validated against the available balance.

Program - 19

Aim:

To write a C++ program to demonstrate the use of constructors and destructors.

Theory:

A constructor is a special member function with the same name as its class and no return type. It is called automatically when an object is created and is commonly used to initialise data members. A destructor has the class name preceded by a tilde and is called automatically when an object is destroyed. Constructors can be overloaded to support different initialisation methods, while a destructor can perform cleanup when an object's lifetime ends.

A constructor has the same name as its class and is called automatically when an object is initialised; it has no return type. A destructor uses the class name preceded by a tilde and is called automatically when the object's lifetime ends. These special member functions are commonly used to initialise and release resources in a predictable order.

Algorithm:

Step - 1 Start the program and define a class containing a constructor, a member function and a destructor.
Step - 2 In the constructor, initialise the object's data members and display a message indicating that construction has occurred.
Step - 3 Create an object in `main()`; its constructor runs automatically when the object is created.
Step - 4 Call the required member function and observe the object's behaviour while it is in scope.
Step - 5 Allow execution to leave the object's scope; the destructor is invoked automatically as the object is destroyed.
Step - 6 Observe the construction and destruction messages in their execution order, then stop.

Code:

```cpp
#include <iostream>
using namespace std;
class Student
{
public:
    // Constructor
    Student()
    {
        cout << "Constructor Called: Object Created." << endl;
    }
    // Destructor
    ~Student()
    {
        cout << "Destructor Called: Object Destroyed." << endl;
    }
    void display()
    {
        cout << "Welcome to C++ Programming!" << endl;
    }
};
int main()
{
    Student s1;     // Constructor is called automatically
    s1.display();
    return 0;       // Destructor is called automatically
}
```

Output:

```
Constructor Called: Object Created.
Welcome to C++ Programming!
Destructor Called: Object Destroyed.
```

Code Explanation:

The class defines a constructor that runs automatically when an object is created and a destructor that runs when the object is destroyed or leaves scope. The program creates objects to demonstrate the order in which these special member functions execute. The displayed messages make object construction and destruction visible.

Conclusion:

The program demonstrated constructor and destructor calls during an object's lifetime. The constructor was used to initialise an object, while the destructor was invoked when the object was destroyed. This practical illustrated automatic object initialisation and cleanup in C++.

Program - 20

Aim:

To write a C++ program to read a complex number and copy it into another using a copy constructor.

Theory:

A copy constructor creates a new object by initialising it from an existing object of the same class. Its usual form is `ClassName(const ClassName &obj)`. Passing the source object by reference avoids attempting to create another copy just to pass the argument, and `const` prevents the constructor from modifying that source object. For a complex number, the real and imaginary parts are copied into the new object.

The copy constructor initialises a distinct new object using the state of an existing object. Taking the source as a constant reference avoids making another copy just to pass the argument and prevents accidental modification of the source. For this class, copying both components preserves the complete complex number.

Algorithm:

Step - 1 Start the program and define class `Complex` with real and imaginary data members.
Step - 2 Provide a parameterised constructor to initialise both components and a copy constructor that accepts a constant reference to another `Complex` object.
Step - 3 Read the real and imaginary parts from the user.
Step - 4 Construct object `c1` using the entered values.
Step - 5 Construct `c2` from `c1`; the copy constructor transfers the source object's real and imaginary components into the new object.
Step - 6 Display both objects in complex-number form and compare their values.
Step - 7 Stop the program.

Code:

```
#include <iostream>
using namespace std;
class Complex
{
    float real, imag;
public:
    // Parameterized constructor
    Complex(float r = 0, float i = 0)
    {
        real = r;
        imag = i;
    }
    // Copy constructor
    Complex(const Complex &c)
    {
        real = c.real;
        imag = c.imag;
    }
    void display()
    {
        cout << real;
        if (imag >= 0)
            cout << " + " << imag << "i";
        else
            cout << " - " << -imag << "i";
        cout << endl;
    }
```

```
};
int main()
{
    float real, imag;
    cout << "Enter real part: ";
    cin >> real;
    cout << "Enter imaginary part: ";
    cin >> imag;
    Complex c1(real, imag);
    Complex c2(c1);        // Copy c1 into c2 using copy constructor
    cout << "\nOriginal Complex Number: ";
    c1.display();
    cout << "Copied Complex Number:   ";
    c2.display();
    return 0;
}
```

Output:

```
Enter real part: 5
Enter imaginary part: 7

Original Complex Number: 5 + 7i
Copied Complex Number:   5 + 7i
```

Code Explanation:

The program defines a class for complex numbers with real and imaginary parts. A copy constructor initialises a new object from an existing object by copying those members. The program creates a second object from the first and displays both values, demonstrating how an object can be initialised from another object of the same class.

Conclusion:

The copy constructor created a new complex-number object by copying the real and imaginary parts from an existing object. The practical demonstrated object initialisation from another object and the use of a const reference parameter in a copy constructor.

Program - 21

Aim:

To write a C++ program to implement function overloading for finding geometric areas.

Theory:

Function overloading allows multiple functions to share the same name when their parameter lists differ in number, type or order. The compiler selects the appropriate overload based on the arguments in the call. In a geometric-area program, overloaded functions can calculate areas for different shapes using the required dimensions. Return type alone cannot distinguish overloaded functions.

Overloaded functions must differ in their parameter lists; changing only the return type would not be enough for the compiler to select an overload. The number and types of arguments distinguish the three area calculations. Each formula assumes valid dimensions, normally non-negative radius, length, breadth, base and height.

Algorithm:

Step - 1 Start the program and define an overloaded `area()` function for a circle, a rectangle and a triangle.
Step - 2 For the circle overload, accept one radius and return `3.14159 * r * r`.
Step - 3 For the rectangle overload, accept length and breadth and return `l * b`.
Step - 4 For the triangle overload, accept base, height and the distinguishing parameter, and return `0.5 * base * h`.
Step - 5 Read the dimensions required for each shape and call the overload whose parameter list matches those arguments.
Step - 6 Display each calculated area with the name of its corresponding shape.
Step - 7 Stop the program.

Code:

```cpp
#include <iostream>
using namespace std;
// Area of circle
float area(float radius)
{
    return 3.14159 * radius * radius;
}
// Area of rectangle
float area(float length, float breadth)
{
    return length * breadth;
}
// Area of triangle (extra int parameter distinguishes this overload)
float area(float base, float height, int)
{
    return 0.5 * base * height;
}
int main()
{
    float r, l, b, base, h;
    cout << "Enter radius of circle: ";
    cin >> r;
    cout << "Area of Circle = " << area(r) << endl;
    cout << "\nEnter length and breadth of rectangle: ";
    cin >> l >> b;
    cout << "Area of Rectangle = " << area(l, b) << endl;
    cout << "\nEnter base and height of triangle: ";
```

```
28      cin >> base >> h;
29      cout << "Area of Triangle = " << area(base, h, 0) << endl;
30      return 0;
31  }
```

Output:

```
1  Enter radius of circle: 5
2  Area of Circle = 78.5397
3
4  Enter length and breadth of rectangle: 10 6
5  Area of Rectangle = 60
6
7  Enter base and height of triangle: 8 5
8  Area of Triangle = 20
```

Code Explanation:

The program overloads a function name for different geometric shapes or parameter lists. Each version receives the dimensions required for its shape and calculates the corresponding area using the appropriate formula. The compiler selects the matching function based on the number and types of arguments, and the results are displayed.

Conclusion:

The overloaded functions calculated geometric areas using different parameter lists. The compiler selected the matching function according to the arguments supplied. This practical demonstrated compile-time function overloading and code reuse through a common function name.

Program - 22

Aim:

To write a C++ program for the addition of two complex numbers using a friend function.

Theory:

A friend function is a non-member function that is granted access to a class's private and protected members. It is declared inside the class using the keyword `friend`, but defined and called like an ordinary function. For complex-number addition, the function accesses the real and imaginary components of both operands and returns or displays their sum. A friend function is not called using an object and the dot operator.

The friend declaration grants the named non-member function access to private members, but it does not make that function a member of the class. Complex-number addition is performed component by component: real parts are added together and imaginary parts are added together. The returned object represents the sum and can be displayed like the inputs.

Algorithm:

Step - 1 Start the program and define class `Complex` with real and imaginary components and a suitable constructor.

Step - 2 Declare `add(Complex, Complex)` as a friend inside the class so it can access private members.

Step - 3 Define the friend function outside the class; add the two real components and separately add the two imaginary components.

Step - 4 Read the components of two complex numbers and construct objects `c1` and `c2`.

Step - 5 Call `add(c1, c2)` and store the returned complex number in `c3`.

Step - 6 Display both input numbers and their sum, preserving the real and imaginary parts in the output.

Step - 7 Stop the program.

Code:

```
#include <iostream>
using namespace std;
class Complex
{
    float real, imag;
public:
    // Constructor to initialize data members
    Complex(float r = 0, float i = 0)
    {
        real = r;
        imag = i;
    }
    // Friend function declaration
    friend Complex add(Complex, Complex);
    void display()
    {
        cout << real << " + " << imag << "i";
    }
};
// Friend function definition
Complex add(Complex c1, Complex c2)
{
    Complex c3;
    c3.real = c1.real + c2.real;
    c3.imag = c1.imag + c2.imag;
```

```
    return c3;
}
int main()
{
    float r1, i1, r2, i2;
    cout << "Enter real and imaginary part of first complex number: ";
    cin >> r1 >> i1;
    cout << "Enter real and imaginary part of second complex number: ";
    cin >> r2 >> i2;
    Complex c1(r1, i1);
    Complex c2(r2, i2);
    Complex c3 = add(c1, c2);   // Addition using friend function
    cout << "\nFirst Complex Number  = ";
    c1.display();
    cout << "\nSecond Complex Number = ";
    c2.display();
    cout << "\nAddition              = ";
    c3.display();
    return 0;
}
```

Output:

```
Enter real and imaginary part of first complex number: 5 3
Enter real and imaginary part of second complex number: 4 2

First Complex Number  = 5 + 3i
Second Complex Number = 4 + 2i
Addition              = 9 + 5i
```

Code Explanation:

The program defines a complex-number class and declares a friend function that can access its private members. The friend function receives the relevant objects, adds their real parts and imaginary parts separately, and returns or displays the resulting complex number. This demonstrates how a non-member function can be granted access to private class data.

Conclusion:

The friend function accessed the complex-number data required to perform addition and produced the combined result. This practical demonstrated how a non-member function can be granted access to private class members and used to implement an operation on objects.

Program - 23

Aim:

To write a C++ program to find the mean of the given numbers using a friend function.

Theory:

The arithmetic mean of a set of values is their sum divided by the number of values: Mean $= \frac{x_1+x_2+\cdots+x_n}{n}$, where n must be greater than zero. A friend function can access private data when the class explicitly grants permission. In this program, the function uses the stored values and count to calculate the mean and display the result.

The mean is a measure of central tendency and is calculated by dividing the sum of all observations by the number of observations. The friend function can read the private array and count directly because the class explicitly grants it access. The calculation requires at least one value to avoid division by zero.

Algorithm:

Step - 1 Start the program and define class `Mean` with a count, an array of values and a member function to read the data.
Step - 2 Declare `calculateMean()` as a friend function so it can access the class's private array and count.
Step - 3 Read the number of values and store each input value in the object using `getData()`.
Step - 4 Inside `calculateMean()`, initialise a sum to zero and add every stored value to it.
Step - 5 Divide the sum by the number of values to obtain the arithmetic mean; the count must be greater than zero.
Step - 6 Return the mean and display it with an explanatory label.
Step - 7 Stop the program.

Code:

```
#include <iostream>
using namespace std;
class Mean
{
    int n;
    float arr[100];
public:
    Mean(int count)
    {
        n = count;
    }
    void getData()
    {
        cout << "Enter " << n << " numbers: ";
        for (int i = 0; i < n; i++)
            cin >> arr[i];
    }
    friend float calculateMean(Mean);
};
// Friend function
float calculateMean(Mean m)
{
    float sum = 0;
    for (int i = 0; i < m.n; i++)
        sum += m.arr[i];
    return sum / m.n;
}
```

```
28 int main()
29 {
30     int n;
31     cout << "Enter number of values: ";
32     cin >> n;
33     Mean m(n);
34     m.getData();
35     cout << "Mean = " << calculateMean(m);
36     return 0;
37 }
```

Output:

```
1 Enter number of values: 5
2 Enter 5 numbers: 10 20 30 40 50
3 Mean = 30
```

Code Explanation:

The program stores the numbers in objects and uses a friend function to access their required values. The function adds the values and divides the sum by the number of values to calculate the mean. The result is displayed, demonstrating friend-function access and the arithmetic steps involved in finding an average.

Conclusion:

The program calculated the arithmetic mean by dividing the sum of the values by their count. The friend function accessed the required class data to perform the calculation. This practical demonstrated the mean formula and the role of friend functions in accessing private members.

Program - 24

Aim:

To write a C++ program to add the integer values of two class objects.

Theory:

Operator overloading gives an existing C++ operator a suitable meaning for a user-defined class. It is implemented through a special function such as `operator+`. When two objects are added, the overloaded function combines their corresponding data members and produces the required result. Operator overloading makes class operations more natural to use, while the operator's meaning should remain intuitive.

The overloaded operator is a member function that defines how the plus sign behaves when its operands are objects of
textttNumber. It does not change addition for built-in types. Returning a new object keeps the inputs unchanged and allows the expression to be written in a natural form.

Algorithm:

Step - 1 Start the program and define class `Number` with an integer data member, a constructor and a display function.
Step - 2 Overload the binary plus operator using `operator+()` so it creates and returns an object containing the sum of the two values.
Step - 3 Create objects `n1` and `n2` with values 10 and 20.
Step - 4 Evaluate `n1 + n2`; the overloaded function adds the values and returns the result object assigned to `n3`.
Step - 5 Call the display function on `n3` to print the calculated sum.
Step - 6 Stop the program.

Code:

```cpp
#include <iostream>
using namespace std;
class Number
{
    int value;
public:
    // Constructor
    Number(int v = 0)
    {
        value = v;
    }
    // Operator overloading for +
    Number operator+(Number obj)
    {
        Number temp;
        temp.value = value + obj.value;
        return temp;
    }
    void display()
    {
        cout << "Sum = " << value << endl;
    }
};
int main()
{
    Number n1(10), n2(20), n3;
    n3 = n1 + n2;
```

```
28      n3.display();
29      return 0;
30  }
```

Output:

```
1  Sum = 30
```

Code Explanation:

The program defines a class whose objects contain numeric data. An overloaded '+' operator or a member function adds the corresponding data from two objects and returns the result as another object. In 'main()', the two input objects are created, added, and the resultant object is displayed.

Conclusion:

The overloaded addition operator combined two class objects according to the operation defined by the class. This practical demonstrated how operator overloading can make operations on user-defined types more readable and natural.

Program - 25

Aim:

To write a C++ program for the addition of two time objects.

Theory:

Adding two time values requires carrying excess units between seconds, minutes and hours. Since 60 seconds make one minute and 60 minutes make one hour, the seconds are added first and any complete groups of 60 are carried into the minutes. The minutes are then normalised in the same way before calculating the final hours. Operator overloading can provide a convenient way to add two time objects.

Normalising seconds and minutes with quotient-and-remainder operations ensures that each resulting lower unit is in the range 0 to 59. The carry from seconds is included before the minute total is normalised, and the resulting minute carry is included in the hours. This representation assumes elapsed time values and does not wrap hours at 24.

Algorithm:

Step - 1 Start the program and define class `Time` with hour, minute and second members, along with input and display functions.

Step - 2 Overload the plus operator to create a result time object from two time objects.

Step - 3 Add the seconds first; carry each complete group of 60 into the minutes and retain the remainder as the result seconds.

Step - 4 Add both minute values and the carried seconds; carry each complete group of 60 into the hours and retain the remainder as result minutes.

Step - 5 Add both hour values and the carry from the minutes to obtain the result hours.

Step - 6 Read two time values, evaluate `t3 = t1 + t2`, and display the result in hours, minutes and seconds.

Step - 7 Stop the program.

Code:

```cpp
#include <iostream>
using namespace std;
class Time
{
    int hours, minutes, seconds;
public:
    void getTime()
    {
        cout << "Enter hours, minutes and seconds: ";
        cin >> hours >> minutes >> seconds;
    }
    void display()
    {
        cout << hours << " hours "
             << minutes << " minutes "
             << seconds << " seconds";
    }
    // Operator overloading for +
    Time operator+(Time t)
    {
        Time temp;
        temp.seconds = seconds + t.seconds;
        temp.minutes = minutes + t.minutes + temp.seconds / 60;
        temp.seconds = temp.seconds % 60;
        temp.hours   = hours + t.hours + temp.minutes / 60;
```

```
        temp.minutes = temp.minutes % 60;
        return temp;
    }
};
int main()
{
    Time t1, t2, t3;
    cout << "Enter first time:\n";
    t1.getTime();
    cout << "Enter second time:\n";
    t2.getTime();
    t3 = t1 + t2;
    cout << "\nFirst Time  = ";
    t1.display();
    cout << "\nSecond Time = ";
    t2.display();
    cout << "\nAddition of Two Times = ";
    t3.display();
    return 0;
}
```

Output:

```
Enter first time:
Enter hours, minutes and seconds: 5 45 50
Enter second time:
Enter hours, minutes and seconds: 3 25 30

First Time  = 5 hours 45 minutes 50 seconds
Second Time = 3 hours 25 minutes 30 seconds
Addition of Two Times = 9 hours 11 minutes 20 seconds
```

Code Explanation:

The program represents time using class data members for hours, minutes and/or seconds. It creates two time objects and adds their components. When a component reaches its valid limit, such as 60 minutes or seconds, the excess is carried into the next component. The normalised result is displayed as the sum of the two times.

Conclusion:

The program added two time objects and accounted for the conversion of excess seconds into minutes and excess minutes into hours. This practical demonstrated object-based arithmetic, unit normalisation and the use of an overloaded operator for time addition.

Program - 26

Aim:

To write a C++ program to design a class having a static member function `showcount()` that displays the number of objects created.

Theory:

A static member function belongs to the class rather than to a particular object and can be called using the class name and scope-resolution operator. It can directly access static data members, but it does not have a `this` pointer and cannot directly access non-static members without an object. A static function such as `showcount()` is suitable for displaying a shared object counter.

Because the counter belongs to the class, every constructor updates the same value. The static function can display this shared value using the class name, without needing an object to call it. This is a direct demonstration of the difference between class-level state and per-object data.

Algorithm:

Step - 1 Start the program and define class `Demo` with a static integer data member `count`.
Step - 2 Define and initialise `Demo::count` outside the class with the value zero.
Step - 3 Increment the shared counter in the constructor so every newly created object contributes to the count.
Step - 4 Implement the static function `showcount()` to display the current shared count.
Step - 5 Create four `Demo` objects in `main()`, causing the counter to increase four times.
Step - 6 Call `Demo::showcount()` using the class name and scope-resolution operator and observe the final count.
Step - 7 Stop the program.

Code:

```cpp
#include <iostream>
using namespace std;
class Demo
{
    static int count;
public:
    Demo()
    {
        count++;
    }
    static void showcount()
    {
        cout << "Number of objects created = " << count << endl;
    }
};
// Definition of static data member
int Demo::count = 0;
int main()
{
    Demo d1;
    Demo d2;
    Demo d3;
    Demo d4;
    Demo::showcount();
    return 0;
}
```

Output:

```
Number of objects created = 4
```

Code Explanation:

The class uses a static data member to maintain a count shared by all objects. Its static member function ‘showcount()‘ displays that shared value and can be called using the class name without creating an object. The program demonstrates the difference between class-wide data and ordinary object members.

Conclusion:

The static member function displayed information shared by the class without requiring it to operate on a particular object. This practical demonstrated how static functions are associated with the class and how they can access static data members.

Program - 27

Aim:

To write a C++ program that uses unary operator (++ / --) overloading.

Theory:

A unary operator acts on one operand. C++ permits operators such as increment ++ and decrement - to be overloaded for user-defined classes. When implemented as a member function, the object on which the operator is used is the operand. The overloaded operation should update the object's data in the manner defined by the program. Prefix and postfix forms can use different function signatures when both are implemented.

A unary operator acts on one object, and an overloaded member operator applies its operation to the calling object. Increment and decrement change the stored value by one, illustrating how operators can provide concise syntax for class behaviour. The example uses the prefix form, whose member function has no dummy parameter.

Algorithm:

Step - 1 Start the program and define class `Number` with an integer value and a constructor for initialisation.
Step - 2 Overload the unary increment operator to increase the calling object's value by one.
Step - 3 Overload the unary decrement operator to decrease the calling object's value by one.
Step - 4 Create an object with an initial value of 10 and display its original value.
Step - 5 Apply the overloaded ++ operator and display the updated value.
Step - 6 Apply the overloaded - operator and display the value again to observe the decrement operation.
Step - 7 Stop the program.

Code:

```
#include <iostream>
using namespace std;
class Number
{
    int value;
public:
    Number(int v)
    {
        value = v;
    }
    // Unary ++ operator overloading
    void operator++()
    {
        ++value;
    }
    // Unary -- operator overloading
    void operator--()
    {
        --value;
    }
    void display()
    {
        cout << "Value = " << value << endl;
    }
};
int main()
{
```

```
    Number n(10);
    cout << "Before increment:" << endl;
    n.display();
    ++n;
    cout << "After increment:" << endl;
    n.display();
    --n;
    --n;
    cout << "After two decrements:" << endl;
    n.display();
    return 0;
}
```

Output:

```
Before increment:
Value = 10
After increment:
Value = 11
After two decrements:
Value = 9
```

Code Explanation:

The class overloads the unary increment (‘++‘) and/or decrement (‘–‘) operator to change an object’s value by one. The operator function updates the relevant data member, and the program displays the value before and after the operation. This shows how unary operators can be given behaviour appropriate to user-defined objects.

Conclusion:

The program demonstrated the overloaded unary operator on an object and applied the operation to its data. This practical showed how a unary operator can be given a suitable meaning for a user-defined type while preserving clear and predictable behaviour.

Program - 28

Aim:

To write a C++ program that uses binary operator overloading on strings.

Theory:

Operator overloading allows a class to define how the binary + operator combines two objects. For a string class, the overloaded operator can concatenate the character sequences held by the two objects and produce a combined string. This provides a natural expression for joining values while keeping the implementation details inside the class. Care must be taken to allocate sufficient storage and terminate character arrays correctly when using manual character-array storage.

String concatenation combines the character sequences in order, placing the second string after the first. The overloaded plus operator returns an object containing the combined text, leaving the original objects available for display. When using fixed-size character arrays, the destination buffer must be large enough for both strings and the terminating null character.

Algorithm:

Step - 1 Start the program and define class `String` with a character array and a constructor to initialise the text.
Step - 2 Overload the binary plus operator to create a new object that contains the concatenation of two strings.
Step - 3 Within the operator function, copy the first string into the result buffer and append the second string using `strcat()`.
Step - 4 Create objects `s1` and `s2` with the two input strings.
Step - 5 Evaluate `s3 = s1 + s2` to invoke the overloaded operator and store the combined text.
Step - 6 Display the two original strings and the combined result.
Step - 7 Stop the program.

Code:

```
#include <iostream>
#include <cstring>
using namespace std;
class String
{
    char str[100];
public:
    // Constructor
    String(const char s[] = "")
    {
        strcpy(str, s);
    }
    // Binary + operator overloading (concatenation)
    String operator+(String s)
    {
        String temp;
        strcpy(temp.str, str);
        strcat(temp.str, s.str);
        return temp;
    }
    void display()
    {
        cout << str << endl;
    }
};
int main()
```

```
{
    String s1("Hello ");
    String s2("World");
    String s3;
    s3 = s1 + s2;
    cout << "First String         : ";
    s1.display();
    cout << "Second String        : ";
    s2.display();
    cout << "Concatenated String : ";
    s3.display();
    return 0;
}
```

Output:

```
First String         : Hello
Second String        : World
Concatenated String : Hello World
```

Code Explanation:

The program defines a class that stores a string and overloads the binary '+' operator. When two objects are added, the operator function combines their stored strings and returns a new object containing the result. The main function creates the objects, applies the operator and displays the combined string.

Conclusion:

The overloaded binary addition operator combined the contents of two string objects into a concatenated result. This practical demonstrated how operator overloading can provide a convenient interface for user-defined string data.

Program - 29

Aim:

To write a C++ program to create a Matrix class and overload the + operator to add two matrix objects.

Theory:

A matrix is a rectangular arrangement of elements organised into rows and columns. A two-dimensional array such as `int a[2][2]` can represent a fixed-size matrix, and nested loops access each element using its row and column indices. Matrix addition is performed by adding elements at corresponding positions. An overloaded + operator can encapsulate this element-by-element operation for matrix objects of compatible dimensions.

Matrix addition is defined only for matrices with matching dimensions, because every entry is added to the entry in the same row and column of the other matrix. Nested loops visit each row-column pair exactly once. The overloaded operator returns a separate matrix containing the result.

Algorithm:

Step - 1 Start the program and define class `Matrix` with a fixed 2 by 2 array and functions to read and display its elements.
Step - 2 Overload the plus operator so it returns a matrix whose elements are the sums of corresponding elements of the two operands.
Step - 3 Read all four elements of the first matrix into object `m1`.
Step - 4 Read all four elements of the second matrix into object `m2`.
Step - 5 Use nested loops in the overloaded operator to add entries at matching row and column positions and store them in the result matrix.
Step - 6 Evaluate `m3 = m1 + m2` and display the first matrix, second matrix and resulting matrix.
Step - 7 Stop the program.

Code:

```
#include <iostream>
using namespace std;
class Matrix
{
    int a[2][2];
public:
    // Input matrix elements
    void getdata()
    {
        for (int i = 0; i < 2; i++)
            for (int j = 0; j < 2; j++)
                cin >> a[i][j];
    }
    // Binary + operator overloading
    Matrix operator+(Matrix m)
    {
        Matrix temp;
        for (int i = 0; i < 2; i++)
            for (int j = 0; j < 2; j++)
                temp.a[i][j] = a[i][j] + m.a[i][j];
        return temp;
    }
    // Display matrix
    void display()
    {
        for (int i = 0; i < 2; i++)
```

```
        {
            for (int j = 0; j < 2; j++)
                cout << a[i][j] << "\t";
            cout << endl;
        }
    }
};
int main()
{
    Matrix m1, m2, m3;
    cout << "Enter elements of first matrix:" << endl;
    m1.getdata();
    cout << "Enter elements of second matrix:" << endl;
    m2.getdata();
    m3 = m1 + m2;     // Add two matrix objects
    cout << "\nFirst Matrix:" << endl;
    m1.display();
    cout << "\nSecond Matrix:" << endl;
    m2.display();
    cout << "\nResultant Matrix:" << endl;
    m3.display();
    return 0;
}
```

Output:

```
Enter elements of first matrix:
1 2
3 4
Enter elements of second matrix:
5 6
7 8

First Matrix:
1   2
3   4

Second Matrix:
5   6
7   8

Resultant Matrix:
6   8
10  12
```

Code Explanation:

The 'Matrix' class stores the elements of a 2×2 matrix in a two-dimensional array. Its overloaded '+' operator uses nested loops to add elements at matching row and column positions and stores the sums in a temporary matrix. The returned matrix is displayed, demonstrating operator overloading for structured data.

Conclusion:

The program added corresponding elements of two matrices and stored the result in a matrix object. This practical demonstrated two-dimensional arrays, nested loops and operator overloading for a matrix operation.

Program - 30

Aim:

To write a C++ program with two classes, `Polar` and `Rectangle`, that represent a point in the polar and rectangular systems and convert from one system to the other.

Theory:

Rectangular coordinates represent a point as (x, y), while polar coordinates represent it using the distance r from the origin and angle θ from the positive x-axis. The conversion formulas are $r = \sqrt{x^2 + y^2}$, $\theta = \text{atan2}(y, x)$, $x = r\cos\theta$, and $y = r\sin\theta$. Trigonometric functions in C++ generally use radians. The `atan2()` function helps determine the correct angle quadrant.

The radius represents the distance from the origin and is always non-negative. The angle describes the direction of the point, and converting between degrees and radians is necessary because C++ trigonometric functions use radians. An inverse tangent calculation should account for the coordinate quadrant, typically by using an appropriate two-argument angle function when available.

Algorithm:

Step - 1 Start the program and define class `Rectangle` for Cartesian coordinates `(x,y)` and class `Polar` for radius and angle.

Step - 2 Declare the classes as friends where needed so the conversion constructors can access the other class's coordinate data.

Step - 3 For rectangular-to-polar conversion, calculate the radius using `sqrt(x*x + y*y)` and calculate the angle using the appropriate inverse tangent function; convert radians to degrees for display if required.

Step - 4 For polar-to-rectangular conversion, convert the angle to radians before calculating `x = r*cos(theta)` and `y = r*sin(theta)`.

Step - 5 Create a rectangular-coordinate object and use the conversion constructor to obtain the corresponding polar object.

Step - 6 Convert the polar object back to rectangular coordinates and display the original and converted values.

Step - 7 Stop the program.

Code:

```cpp
#include <iostream>
#include <cmath>
using namespace std;
class Polar;
class Rectangle
{
    float x, y;
public:
    Rectangle(float a = 0, float b = 0)
    {
        x = a;
        y = b;
    }
    Rectangle(Polar p);   // Conversion constructor (Polar -> Rectangle)
    friend class Polar;
    void display()
    {
        cout << "Rectangular Coordinates = (" << x << ", " << y << ")" << endl;
    }
};
class Polar
```

```
{
    float r, theta;
public:
    Polar(float radius = 0, float angle = 0)
    {
        r = radius;
        theta = angle;
    }
    Polar(Rectangle rec)  // Conversion constructor (Rectangle -> Polar)
    {
        r = sqrt(rec.x * rec.x + rec.y * rec.y);
        theta = atan2(rec.y, rec.x) * 180 / 3.14159;
    }
    friend class Rectangle;
    void display()
    {
        cout << "Polar Coordinates = (" << r << ", " << theta << " degrees)" << endl;
    }
};
Rectangle::Rectangle(Polar p)
{
    float angle = p.theta * 3.14159 / 180;
    x = p.r * cos(angle);
    y = p.r * sin(angle);
}
int main()
{
    Rectangle R1(3, 4);
    cout << "Original Rectangle:\n";
    R1.display();
    Polar P1 = R1;            // Rectangle -> Polar
    cout << "\nRectangle to Polar:\n";
    P1.display();
    Rectangle R2 = P1;        // Polar -> Rectangle
    cout << "\nPolar to Rectangle:\n";
    R2.display();
    return 0;
}
```

Output:

```
Original Rectangle:
Rectangular Coordinates = (3, 4)

Rectangle to Polar:
Polar Coordinates = (5, 53.1301 degrees)

Polar to Rectangle:
Rectangular Coordinates = (3, 4)
```

Code Explanation:

The program uses classes to represent polar coordinates ‘(r, theta)‘ and rectangular coordinates ‘(x, y)‘. It applies the formulas ‘x = r cos(theta)‘, ‘y = r sin(theta)‘, ‘r = sqrt(x*x + y*y)‘ and ‘theta = atan2(y, x)‘ as appropriate. Angles are converted between degrees and radians where required. Conversion constructors or class functions transfer values between the two representations.

Conclusion:

The program converted between rectangular and polar coordinates using the appropriate mathematical relationships. This practical demonstrated coordinate representation, square-root and trigonometric functions, and the importance of using the correct angle units.

Program - 31

Aim:

To write a C++ program to implement single inheritance.

Theory:

Inheritance allows a derived class to acquire accessible data and functions from a base class. In single inheritance, one derived class inherits from one base class. It supports code reuse and represents an *is-a* relationship where appropriate. Public inheritance preserves the public interface of the base class as public in the derived class, while private members of the base class cannot be accessed directly by derived-class member functions.

Public single inheritance allows the derived class to use accessible features of one base class while adding its own data and functions. This avoids rewriting the student identity code in the result class. The protected members are available to derived-class functions but are not directly accessible from unrelated code.

Algorithm:

Step - 1 Start the program and define base class `Student` with student name and roll number and functions to read and display these details.

Step - 2 Use protected access for the inherited data so the derived class can use it while keeping it unavailable to unrelated code.

Step - 3 Define class `Result` using public single inheritance from `Student`; add the marks data and functions to read and display the result.

Step - 4 Create a `Result` object in `main()`.

Step - 5 Call the inherited student-input function and the derived-class function for entering marks.

Step - 6 Display the student details and marks together to demonstrate reuse of base-class functionality.

Step - 7 Stop the program.

Code:

```
#include <iostream>
#include <string>
using namespace std;
// Base Class
class Student
{
protected:
    int rollNo;
    string name;
public:
    void getStudentData()
    {
        cout << "Enter Roll Number: ";
        cin >> rollNo;
        cout << "Enter Name: ";
        cin >> name;
    }
    void displayStudentData()
    {
        cout << "Roll Number: " << rollNo << endl;
        cout << "Name: " << name << endl;
    }
};
// Derived Class
```

```
25 class Result : public Student
26 {
27 private:
28     float marks;
29 public:
30     void getResult()
31     {
32         cout << "Enter Marks: ";
33         cin >> marks;
34     }
35     void displayResult()
36     {
37         displayStudentData();
38         cout << "Marks: " << marks << endl;
39     }
40 };
41 int main()
42 {
43     Result r;
44     r.getStudentData();
45     r.getResult();
46     cout << "\n--- Student Result ---\n";
47     r.displayResult();
48     return 0;
49 }
```

Output:

```
1 Enter Roll Number: 101
2 Enter Name: Sanket
3 Enter Marks: 85
4
5 --- Student Result ---
6 Roll Number: 101
7 Name: Sanket
8 Marks: 85
```

Code Explanation:

The program defines a base class containing common data or functions and a derived class that inherits those members. An object of the derived class can use the accessible inherited functionality as well as its own members. The program calls the relevant functions and displays the results, demonstrating single inheritance from one base class.

Conclusion:

The program demonstrated single inheritance by deriving a class from one base class and using accessible inherited features. This practical illustrated code reuse and the relationship between base and derived classes.

Program - 32

Aim:

To write a C++ program to implement multiple inheritance.

Theory:

Multiple inheritance occurs when a derived class inherits from two or more base classes. The base classes are listed in the derived-class declaration, separated by commas and accompanied by access specifiers. The derived class can combine accessible features from each base class. If base classes contain members with the same name, the program may need scope resolution to identify the intended member.

The derived class combines features from two separate base classes, allowing it to reuse both student identity and marks functionality. Each base class handles its own data, while the result class presents the combined information. Multiple inheritance is useful when a type naturally needs capabilities from more than one source class.

Algorithm:

Step - 1 Start the program and define base class `Student` with student identity data and a function to read it.
Step - 2 Define a second base class `Marks` with marks data and a function to input the marks.
Step - 3 Define class `Result` inheriting publicly from both `Student` and `Marks`.
Step - 4 Add a `display()` function to `Result` that can use the accessible data and show the combined result.
Step - 5 Create a `Result` object, then call the inherited input functions to collect student information and marks.
Step - 6 Call `display()` to show the information supplied by both base classes.
Step - 7 Stop the program.

Code:

```
#include <iostream>
#include <string>
using namespace std;
// Base Class 1
class Student
{
protected:
    int rollNo;
    string name;
public:
    void getStudentData()
    {
        cout << "Enter Roll Number: ";
        cin >> rollNo;
        cout << "Enter Name: ";
        cin >> name;
    }
};
// Base Class 2
class Marks
{
protected:
    float marks;
public:
    void getMarks()
    {
```

```
        cout << "Enter Marks: ";
        cin >> marks;
    }
};
// Derived Class
class Result : public Student, public Marks
{
public:
    void display()
    {
        cout << "\n--- Student Result ---" << endl;
        cout << "Roll Number: " << rollNo << endl;
        cout << "Name: " << name << endl;
        cout << "Marks: " << marks << endl;
    }
};
int main()
{
    Result r;
    r.getStudentData();
    r.getMarks();
    r.display();
    return 0;
}
```

Output:

```
Enter Roll Number: 101
Enter Name: Naman
Enter Marks: 85

--- Student Result ---
Roll Number: 101
Name: Naman
Marks: 85
```

Code Explanation:

The program defines a derived class that inherits from more than one base class. Each base class contributes its own members, and the derived class can access the inherited members according to their access specifiers. An object of the derived class invokes the available functions and displays the combined information, demonstrating multiple inheritance.

Conclusion:

The program demonstrated multiple inheritance by combining accessible features from more than one base class in a derived class. This practical illustrated how a class can reuse functionality from multiple parent classes and how name conflicts may need to be resolved.

Program - 33

Aim:

To write a C++ program to create a file using a constructor function.

Theory:

File handling allows a C++ program to store information beyond the lifetime of the program. The `<fstream>` header provides stream classes including `ofstream` for writing and `ifstream` for reading files. An output file stream can create a file and write text to it. A constructor can perform this operation when an object is created, and the stream should be checked to confirm that the file opened successfully.

An output file stream connects the program to a file so written information can persist after the program finishes. Opening the file in the constructor ties the resource's lifetime to the object's lifetime, and closing it in the destructor ensures buffered output is flushed and the file is released. Stream status should be checked in robust file-handling code.

Algorithm:

Step - 1 Start the program and define class `FileHandler` with an `ofstream` member for file output.
Step - 2 In the constructor, open the specified file and check whether the stream opened successfully; display the appropriate status message.
Step - 3 Implement `write()` to send the required text lines to the open file stream.
Step - 4 In `main()`, create a `FileHandler` object so its constructor opens the file automatically.
Step - 5 Call the write function for the required lines and display a message confirming that data was written.
Step - 6 When the object goes out of scope, let the destructor close the file stream and display the closing message.
Step - 7 Stop the program.

Code:

```
#include <iostream>
#include <fstream>
#include <string>
using namespace std;
class FileHandler
{
    ofstream file;
public:
    // Constructor creates (opens) the file
    FileHandler(const string &filename)
    {
        file.open(filename);
        if (file.is_open())
            cout << "File \"" << filename << "\" created successfully." << endl
    ;
        else
            cout << "Error: unable to create file." << endl;
    }
    void write(const string &text)
    {
        if (file.is_open())
            file << text << endl;
    }
    // Destructor closes the file
    ~FileHandler()
    {
```

```
        if (file.is_open())
        {
            file.close();
            cout << "File closed." << endl;
        }
    }
};
int main()
{
    FileHandler f("sample.txt");
    f.write("This file was created using a constructor.");
    f.write("Object Oriented Programming Lab.");
    cout << "Data written to file." << endl;
    return 0;
}
```

Output:

```
File "sample.txt" created successfully.
Data written to file.
File closed.
```

Code Explanation:

The program defines a ‘FileHandler‘ class whose constructor opens or creates a file using an output file stream. The program writes text to the file and displays status messages. When the object goes out of scope, its destructor closes the file, demonstrating resource management through object lifetime and file handling with ‘fstream‘.

Conclusion:

The program used a constructor to initiate file creation and write data through a file stream. This practical demonstrated basic file output and how file handling can preserve information outside the program’s execution.

Program - 34

Aim:

To write a C++ program to demonstrate the use of templates.

Theory:

Templates support generic programming by allowing functions and classes to operate with different data types without duplicating their definitions. A function template defines a general operation using a type parameter, while a class template defines a class whose members use a type parameter. The compiler generates suitable versions when the template is instantiated with types such as integers or floating-point values. Template arguments must support the operations used by the template body.

Templates are checked by the compiler when they are instantiated with a particular type. The operations used in the template body must be supported by that type; for example, the maximum function requires comparison, while the calculator requires addition and multiplication. This allows reusable code while retaining compile-time type checking.

Algorithm:

Step - 1 Start the program and define a function template `maximum(T a, T b)` that returns the greater of two values of the same type.
Step - 2 Call the function template with integer, floating-point and character arguments and display each returned maximum.
Step - 3 Define class template `Calculator<T>` with data members for two values of type `T`.
Step - 4 Implement the constructor to initialise both values and member functions to return their sum and product.
Step - 5 Create an integer calculator object and call its addition and multiplication functions.
Step - 6 Create a floating-point calculator object and call the same functions to demonstrate reuse of the class template for another type.
Step - 7 Display all results and stop the program.

Code:

```
#include <iostream>
using namespace std;
// Function template
template <class T>
T maximum(T a, T b)
{
    return (a > b) ? a : b;
}
// Class template
template <class T>
class Calculator
{
    T a, b;
public:
    Calculator(T x, T y)
    {
        a = x;
        b = y;
    }
    T add()      { return a + b; }
    T multiply() { return a * b; }
};
int main()
{
```

```
    cout << "Max of 10 and 25      = " << maximum(10, 25) << endl;
    cout << "Max of 3.5 and 2.1    = " << maximum(3.5, 2.1) << endl;
    cout << "Max of 'a' and 'z'    = " << maximum('a', 'z') << endl;
    Calculator<int> c1(6, 4);
    cout << "\nInteger Calculator:" << endl;
    cout << "Sum     = " << c1.add() << endl;
    cout << "Product = " << c1.multiply() << endl;
    Calculator<float> c2(2.5, 1.5);
    cout << "\nFloat Calculator:" << endl;
    cout << "Sum     = " << c2.add() << endl;
    cout << "Product = " << c2.multiply() << endl;
    return 0;
}
```

Output:

```
Max of 10 and 25      = 25
Max of 3.5 and 2.1    = 3.5
Max of 'a' and 'z'    = z

Integer Calculator:
Sum     = 10
Product = 24

Float Calculator:
Sum     = 4
Product = 3.75
```

Code Explanation:

The function template ‘maximum()‘ uses a placeholder type ‘T‘ so the same function can compare values of different compatible types. The ‘Calculator<T>‘ class template stores two values and provides ‘add()‘ and ‘multiply()‘ operations. Objects such as ‘Calculator<int>‘ and ‘Calculator<float>‘ instantiate the class for different types, demonstrating generic code reuse.

Conclusion:

The program demonstrated both function and class templates with more than one data type. The template function selected the larger value, while the template class performed arithmetic operations. This practical illustrated generic programming, reusable definitions and compile-time template instantiation.